NAME-ANURAG SURVE

ROLL NO.03

OPERATING SYSTEM MINI-PROJECT

- **Title:** Disk Scheduling Simulator: Implementation and Performance Comparison of Disk Scheduling Algorithms.

- **Problem statement:**

In an operating system, multiple processes may request data from different disk tracks. Servicing these requests inefficiently can increase disk-head movement and access time. The purpose of this project is to simulate different disk scheduling algorithms, calculate the total head movement, and compare their performance using the same set of disk requests.

- **Objectives:**

  - Understand disk scheduling in an operating system.
  - Implement FCFS, SSTF, SCAN, and C-SCAN algorithms.
  - Accept disk requests and the initial head position from the user.
  - Display the order in which requests are serviced.
  - Calculate total head movement and average head movement per request.
  - Compare the algorithms using a common input.

- **Hardware and software requirements:**

Hardware	Software
Computer or laptop	Windows, Linux, or macOS

Minimum 2 GB RAM	Python 3.10 or later
Keyboard and monitor	VS Code, IDLE, or another Python editor
Approximately 100 MB free storage	No external Python libraries required

- **Algorithm:**

Step 1: Start the program.

Step 2: Input the disk size, disk request queue, initial head position, and direction of movement.

Step 3: Validate the input values. If invalid, display an error message and stop.

Step 4: Select and execute the disk scheduling algorithms:

- FCFS: Service requests in the order they are received.
- SSTF: Select the pending request nearest to the current head position.
- SCAN: Move the head in the selected direction, service requests, reach the disk boundary, and reverse direction.
- C-SCAN: Move the head in the selected direction, service requests, reach the boundary, wrap to the opposite end, and continue in the same direction.

Step 5: Record the sequence of head positions for each algorithm.

Step 6: Calculate total head movement using:

$$\text{Total Movement} = \sum_{i=1}^{n} |H_i - H_{i-1}|$$

Step 7: Calculate average head movement per request:

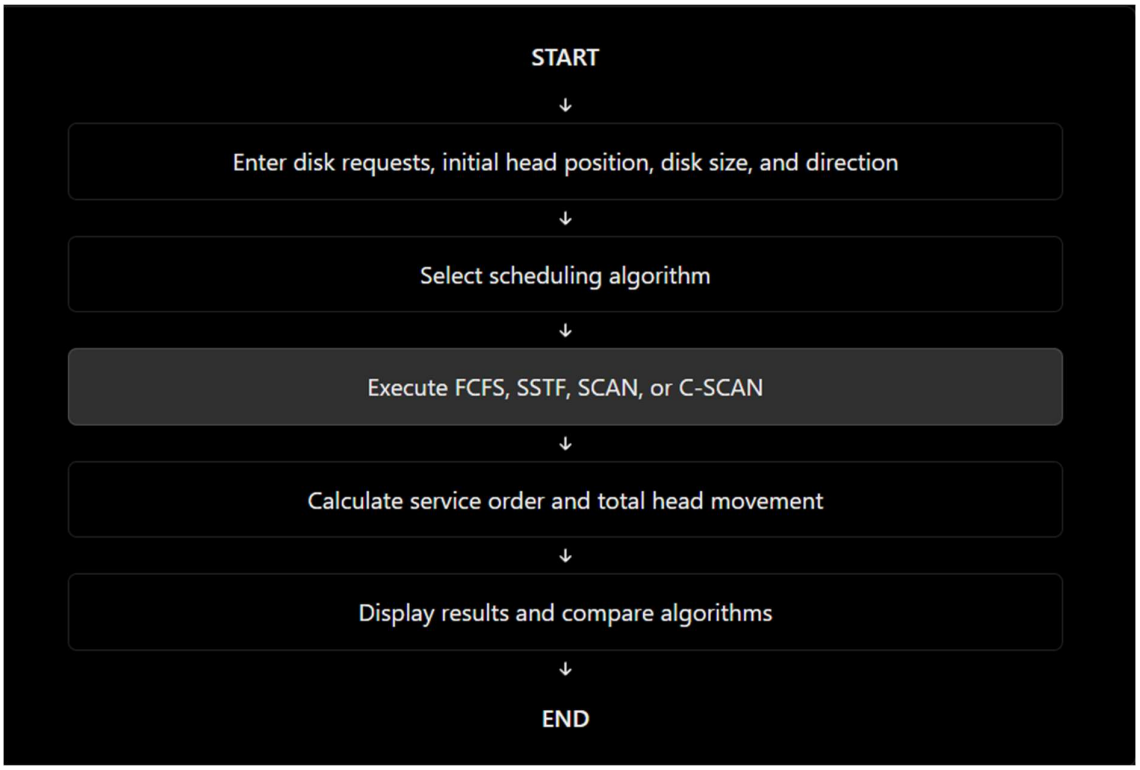
$$\text{Average Movement} = \frac{\text{Total Head Movement}}{\text{Number of Requests}}$$

Step 8: Display the service order, head movement, and average movement for each algorithm.

Step 9: Compare the results and identify the algorithm with the lowest total head movement for the given input.

Step 10: Stop the program.

- **Flowchart:**



- **SOURCE CODE:**

  INPUT:

```python

def fcfs(requests, head, disk_size, direction):
    return requests[:]


def sstf(requests, head, disk_size, direction):
    pending = requests[:]
    order = []
    current = head

    while pending:
        nearest = min(pending, key=lambda track: abs(track - current))
        order.append(nearest)
        pending.remove(nearest)
        current = nearest

    return order


def scan(requests, head, disk_size, direction):
    left = sorted([r for r in requests if r < head])
    right = sorted([r for r in requests if r >= head])

    if direction == "left":
        return list(reversed(left)) + right
    return right + list(reversed(left))

```

```python

def cscan(requests, head, disk_size, direction):
    left = sorted([r for r in requests if r < head])
    right = sorted([r for r in requests if r >= head])

    if direction == "right":
        return right + left
    return list(reversed(left)) + list(reversed(right))


def calculate_movement(order, head, algorithm, disk_size, direction):
    current = head
    total = 0
    path = [head]

    if algorithm == "SCAN":
        if direction == "right":
            right = sorted([r for r in order if r >= head])
            left = sorted([r for r in order if r < head], reverse=True)
            path = [head] + right
            if left:
                if path[-1] != disk_size - 1:
                    path.append(disk_size - 1)
                path += left
        else:
            left = sorted([r for r in order if r < head], reverse=True)
            right = sorted([r for r in order if r >= head])
            path = [head] + left
            if right:
                if path[-1] != 0:
```

```
60
61      elif algorithm == "C-SCAN":
62          if direction == "right":
63              right = sorted([r for r in order if r >= head])
64              left = sorted([r for r in order if r < head])
65              path = [head] + right
66              if left:
67                  if path[-1] != disk_size - 1:
68                      path.append(disk_size - 1)
69                  path.append(0)
70                  path += left
71          else:
72              left = sorted([r for r in order if r < head], reverse=True)
73              right = sorted([r for r in order if r >= head], reverse=True)
74              path = [head] + left
75              if right:
76                  if path[-1] != 0:
77                      path.append(0)
78                  path.append(disk_size - 1)
79                  path += right
80      else:
81          path = [head] + order
82
83      for i in range(1, len(path)):
84          total += abs(path[i] - path[i - 1])
85
86      return total, path
87
88
89  def main():
```

```
88
89  def main():
90      print("=== DISK SCHEDULING SIMULATOR ===")
91      try:
92          disk_size = int(input("Enter number of tracks (e.g. 200): "))
93          if disk_size < 2:
94              raise ValueError("Disk size must be at least 2.")
95
96          requests = list(map(
97              int, input("Enter disk requests separated by spaces: ").split()
98          ))
99          head = int(input("Enter initial head position: "))
100         direction = input("Direction (left/right): ").strip().lower()
101
102         if not requests:
103             raise ValueError("Enter at least one request.")
104         if not 0 <= head < disk_size:
105             raise ValueError("Head position is outside disk limits.")
106         if any(r < 0 or r >= disk_size for r in requests):
107             raise ValueError("A request is outside disk limits.")
108         if direction not in ("left", "right"):
109             raise ValueError("Direction must be left or right.")
110
111     except ValueError as error:
112         print("Invalid input:", error)
113         return
114
115     algorithms = [
116         ("FCFS", fcfs),
117         ("SSTF", sstf),
```

```
        ("FCFS", fcfs),
        ("SSTF", sstf),
        ("SCAN", scan),
        ("C-SCAN", cscan),
    ]

    results = []
    for name, algorithm in algorithms:
        order = algorithm(requests, head, disk_size, direction)
        movement, path = calculate_movement(
            order, head, name, disk_size, direction
        )
        average = movement / len(requests)
        results.append((name, order, movement, average, path))

    print("\n--- RESULTS ---")
    for name, order, movement, average, path in results:
        print(f"\n{name}")
        print("Service order:", " -> ".join(map(str, order)))
        print("Head path:    ", " -> ".join(map(str, path)))
        print("Total head movement:", movement, "tracks")
        print(f"Average movement per request: {average:.2f} tracks")

    best = min(results, key=lambda result: result[2])
    print("\n--- COMPARISON ---")
    for name, _, movement, _, _ in results:
        print(f"{name}: {movement} tracks")
    print("Least head movement for this input:", best[0])
```

```
        print(f"{name}: {movement} tracks")
    print("Least head movement for this input:", best[0])


if __name__ == "__main__":
    main()

```

OUTPUT:

```
=== DISK SCHEDULING SIMULATOR ===
Enter number of tracks (e.g. 200): 200
Enter disk requests separated by spaces: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Direction (left/right): RIGHT

--- RESULTS ---

FCFS
Service order: 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Head path:     53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total head movement: 640 tracks
Average movement per request: 80.00 tracks

SSTF
Service order: 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Head path:     53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total head movement: 236 tracks
Average movement per request: 29.50 tracks

SCAN
Service order: 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 37 -> 14
Head path:     53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
Total head movement: 331 tracks
Average movement per request: 41.38 tracks
```

```
C-SCAN
Service order: 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 14 -> 37
Head path:     53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> 37
Total head movement: 382 tracks
Average movement per request: 47.75 tracks

--- COMPARISON ---
FCFS: 640 tracks
SSTF: 236 tracks
SCAN: 331 tracks
C-SCAN: 382 tracks
Least head movement for this input: SSTF
```

- **Result:**

  The Disk Scheduling Simulator was successfully implemented using Python. The FCFS, SSTF, SCAN, and C-SCAN disk scheduling algorithms were simulated and tested with sample disk requests.

  The program calculated the service order, total head movement, and average head movement per request for each algorithm. For the sample input, SSTF produced the least total head movement of 236 tracks.

- **Future Scope:**

- Add a graphical user interface.
- Display a Gantt-style head movement chart.
- Add LOOK and C-LOOK algorithms.
- Allow users to save and export comparison results.
- Include additional performance measures.

➢ Conclusion:

The Disk Scheduling Simulator successfully demonstrates FCFS, SSTF, SCAN, and C-SCAN disk scheduling algorithms. It calculates and compares total head movement for a common set of requests. The results show that the amount of movement depends on the selected algorithm and input. The project improves understanding of operating system disk management and provides a foundation for further development.